Inventiq

Participant Details

Full name - Aditi Tiwari

Team Name - Team Dhatri

Email ID - aditirenutiwari@gmail.com

Introduction

Inventiq is an integrated inventory and operations management system developed for businesses dealing with electronic components and finished electronic goods. The system was built to address the limitations of fragmented, manual, and non-synchronized operational workflows by providing **a centralized digital platform for inventory tracking, procurement, sales order handling, manufacturing management, reporting, and decision support**. The project aligns with the SRS by focusing on operational control, traceability, real-time visibility, and a structured workflow across the major business modules.

System Overview

It integrates:

- Inventory & products
- Sales & purchase orders
- Manufacturing (WIP batches)
- Customers & suppliers
- Activity log
- Real-time dashboard

It covers the full lifecycle — ***from stock entry → order creation → procurement → production → dispatch → analysis***. Unlike basic CRUD systems, Inventiq acts as an operational control system, giving real-time visibility and decision support, not just data storage.

Key Features Implemented

- **Product and Inventory Management** - Full CRUD support for products, inventory visibility, stock quantity tracking, low-stock awareness, and detailed product records.
- **Sales Order Management** - Structured sales workflow with the stages *Quotation → Packing → Dispatch → History*, along with create, update, delete, and status progression support.

- **Purchase Order Management** - Structured purchase workflow with the stages *Quotations Received → Paid/Unpaid → Order Completion → History*, including full operational handling of purchase records.
- **Manufacturing / Work-in-Progress Management** - Batch-based manufacturing management with creation, tracking, editing, completion handling, and operational queue visibility.
- **Customer and Supplier Management** - Dedicated pages for customers and suppliers with real backend data integration and structured operational display.
- **History and Reporting Module** - Centralized history tracking with filtering, queue-based record review, CSV export, and PDF-style export support.
- **AI Product Chatbot** - Gemini-powered chatbot integrated with the product database to answer product-related queries using live backend data.
- **Live Dashboard Analytics** - A dashboard powered by Recharts showing: sales vs purchases trend operational status pipelines revenue concentration by top customers inventory health overview customer and supplier contribution comparisons
- **Real-Time Synchronization** - Live system updates using Server-Sent Events so changes in products, orders, manufacturing, and history are reflected across the application.
- **Authentication and Session Control** - Secure login/logout flow, protected routes, token-based authentication, and enforcement of a maximum of five concurrent active users.
- **Responsive User Interface** - Control-room style UI optimized for full-screen and smaller-screen usage, with improved scrolling, dialogs, queue layouts, and module consistency.
- **Seeded Business Dataset** - A realistic electronics-focused demo dataset including products, customers, suppliers, sales orders, purchase orders, manufacturing batches, and history entries.
- **Deployment Readiness** - Docker, Docker Compose, environment-based secret handling, and cloud deployment planning for production-style hosting credibility.

Architecture / Technology Stack

Inventiq uses a full-stack client-server architecture.

- **Frontend**: React, TypeScript, Vite, Tailwind CSS, Recharts
- **Backend**: Node.js, Express, TypeScript, Prisma ORM
- **Database**: PostgreSQL
- **Session Management**: Redis
- **Realtime Updates**: Server-Sent Events (SSE)
- **AI Integration**: Gemini API for the product chatbot
- **Deployment Support**: Digital Ocean,Docker, Docker Compose

How the System Addresses the Specific Problem Statement ?

Inventiq addresses the problem of fragmented and inefficient inventory and operations management by bringing products, sales, purchases, manufacturing, customers, suppliers, and history into one connected system.

It solves this by:

- centralizing operational data in a single platform
- enforcing structured workflows for sales and purchase stages
- providing real-time visibility into operational changes
- offering dashboard analytics for better decision-making
- supporting manufacturing and inventory tracking together
- adding an AI chatbot for quick product-related queries using live database data

This helps reduce manual errors, improve coordination between business functions, and make inventory and operations management more efficient and reliable.

Major Critical Issues Tackled as per the SRS

- Dashboard loading in < 2sec
- Provided Transport Layer Security
- Real-Time Operational Synchronization through SSE and cloud
- End-to-End Operational Workflow Integration
- Exact Sales Workflow Implementation
- Manufacturing / Work-in-Progress Management as core business function
- Structured Dashboard and Monitoring Layer

Instructions to Run or Access the Application :-

The application has been deployed on a DigitalOcean server and can be accessed through the server's public IP address.

Open the application in a browser using: **64.227.174.230**

Login Credentials

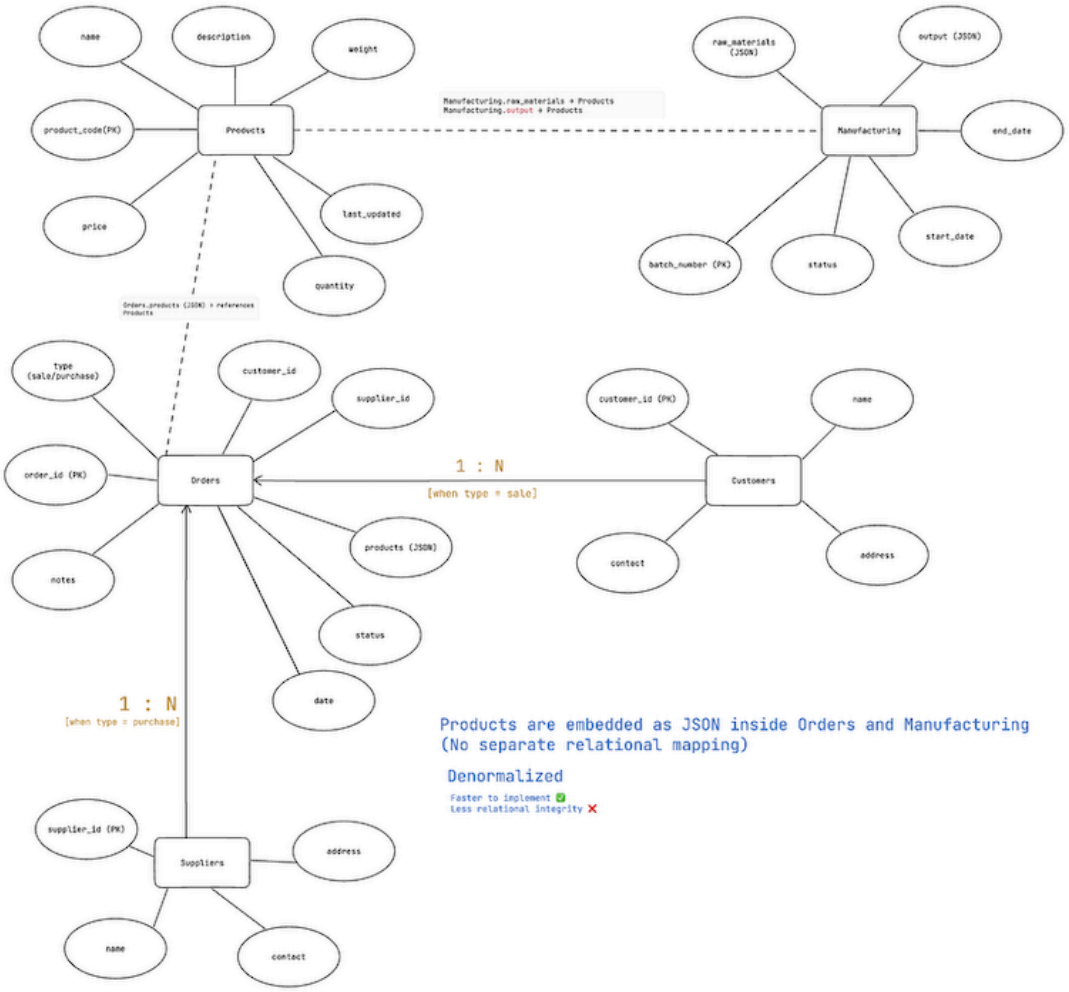
Use the following default credentials:

Username: aditi_admin

Password: Inventiq@123

How to Pull the Project from GitHub and Run It on the Server ➖

1. Connect to the server using SSH: ssh root@<server-ip>
2. Clone the project from GitHub:
   git clone **<https://github.com/AditiT30/inventiq>**
   cd inventiq
3. For future updates, pull the latest code: git pull origin main
4. Create the deployment environment file: cp .env.deploy.example .env.deploy nano .env.deploy
5. Add the required environment values in .env.deploy, such as:
6. JWT_SECRET APP_USER APP_PASS_HASH GEMINI_API_KEY
7. Start the full application stack: docker compose --env-file .env.deploy -f docker-compose.deploy.yml up --build -d
8. Open the application in the browser: http://<server-ip>:8080
9. To populate the database with demo data, run: docker compose --env-file .env.deploy -f docker-compose.deploy.yml exec backend npm run seed
10. This setup starts the frontend, backend, PostgreSQL, and Redis together using Docker Compose.


name
description
weight
product_code(PK)
Products
price
last_updated
quantity
Manufacturing.raw_materials → Products
Manufacturing.output → Products
raw_materials (JSON)
output (JSON)
Manufacturing
end_date
batch_number (PK)
status
start_date
Orders.products (JSON) → references Products
type (sale/purchase)
customer_id
supplier_id
order_id (PK)
Orders
1 : N
[when type = sale]
customer_id (PK)
name
Customers
contact
address
products (JSON)
notes
status
date
1 : N
[when type = purchase]
Products are embedded as JSON inside Orders and Manufacturing
(No separate relational mapping)
Denormalized
Faster to implement ✅
Less relational integrity ❌
supplier_id (PK)
Suppliers
address
name
contact

ER DIAGRAM